

PHASE 16 — CRYPTOGRAPHIC KNOWLEDGE GRAPH, DEPENDENCY MAPPING & BLAST-RADIUS ANALYSIS

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

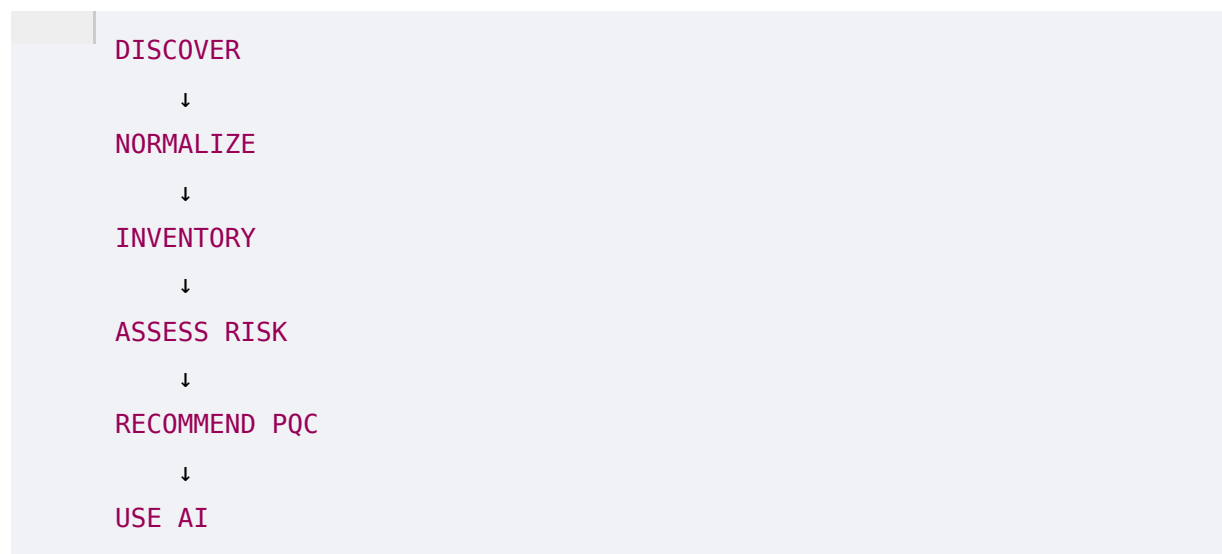
Theme: Blockchain & Cybersecurity

Phase: 16 of 22

Status: Enterprise Cryptographic Relationship & Dependency Intelligence

1. Purpose of This Phase

By now, ECDAT has learned how to:



But there is still a major missing capability.

Suppose ECDAT discovers:

```
OpenSSL
```

and determines:

```
OpenSSL contains quantum-relevant cryptographic usage.
```

That alone isn't enough.

The organization needs to know:

What depends on OpenSSL?

Which applications use it?

Which certificates depend on those applications?

Which business processes depend on those applications?

What happens if OpenSSL is upgraded?

How many systems are affected by migrating one shared cryptographic component?

This is the purpose of the:

Cryptographic Knowledge Graph.

2. Core Idea

Instead of storing ECDAT as a flat table:

```
Asset 1
Asset 2
Asset 3
Asset 4
```

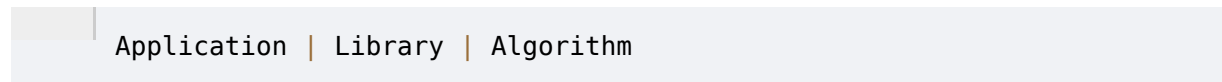
we create relationships:

```
Application
|
|— uses → Library
|           |
|           |— implements → Algorithm
|
|— presents → Certificate
|
|— protects → Data
|
|— supports → Business Process
```

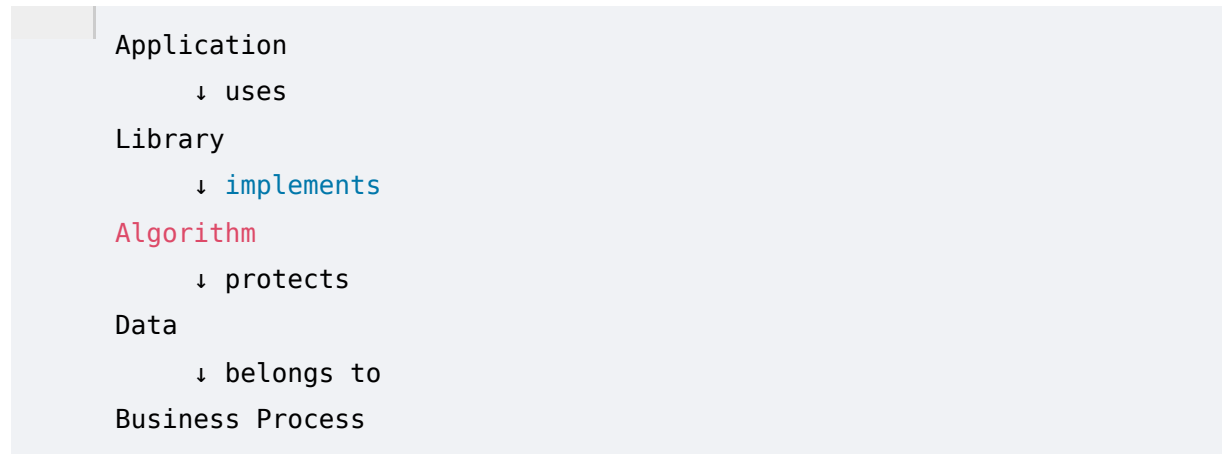
Now ECDAT can reason about the enterprise as a connected system.

3. Why a Graph?

Traditional database:

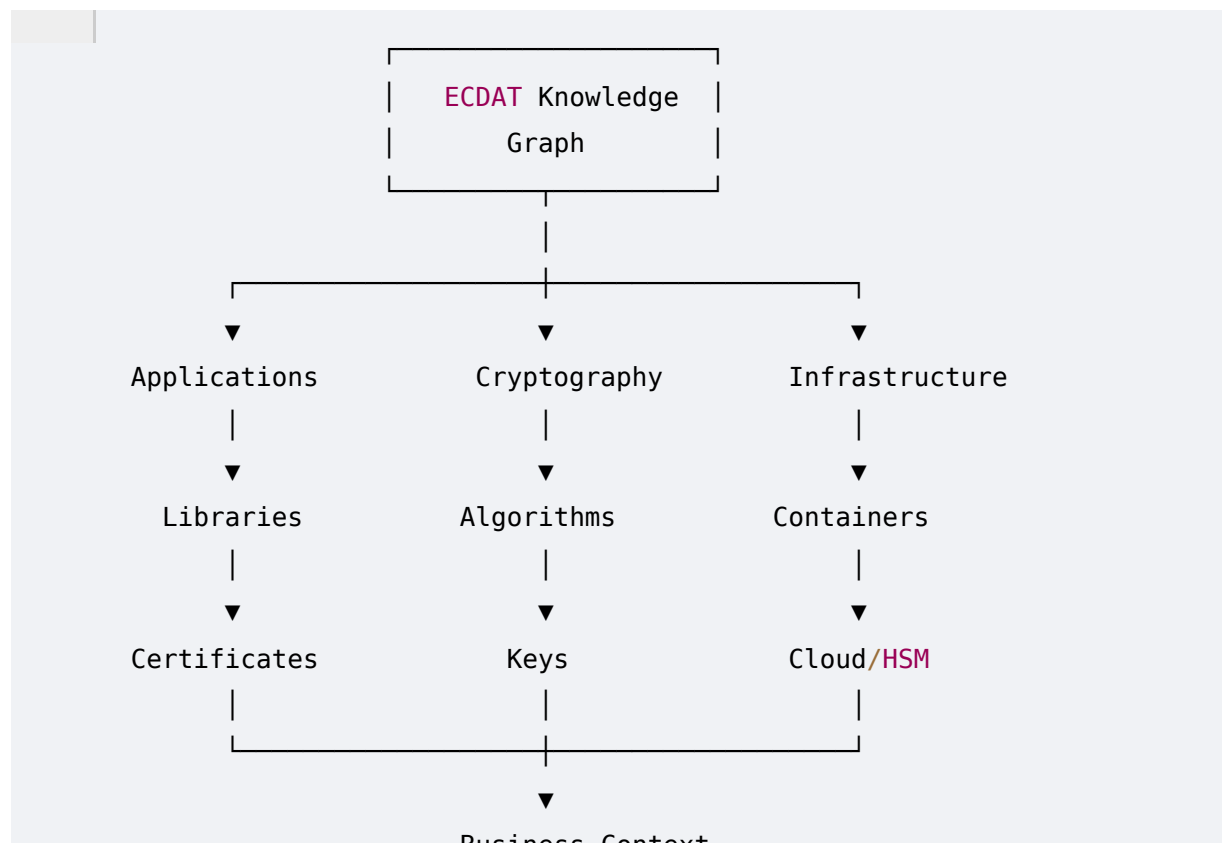


Graph:



The second representation is much better for dependency analysis.

4. Graph Architecture



5. What Is a Node?

A node represents an entity.

Examples:

```
Application
Repository
File
Function
Library
Package
Algorithm
Key
Certificate
Protocol
Container
Host
Cloud Service
HSM
KMS
Data Asset
Business Process
Team
```

6. What Is an Edge?

An edge represents a relationship.

Examples:

```
Application
└─ USES → Library

Library
└─ IMPLEMENTS → Algorithm

Application
└─ PRESENTS → Certificate

Certificate
└─ CONTAINS → Public Key

Public Key
└─ USES → Algorithm
```

7. Why Relationships Matter

Suppose:

```
Algorithm = RSA
Risk = High
```

That is useful.

But:

```
RSA
  ↓
Library X
  ↓
800 applications
  ↓
120 critical services
  ↓
40 business processes
```

is much more useful.

That tells us:

RSA migration has enterprise-wide impact.

8. Graph Node Taxonomy

ECDAT should maintain a controlled node vocabulary.

Software

```
Application
Service
Repository
Package
Library
Binary
Function
Module
```

Cryptography

- Algorithm
- Parameter Set
- Key
- Certificate
- Cipher Suite
- Protocol
- Crypto Provider
- HSM
- KMS

Infrastructure

- Container
- Host
- VM
- Cluster
- Cloud Service
- Endpoint
- Network

Business

- Data Asset
- Business Process
- Business Unit
- Team
- Owner
- Compliance Boundary

9. Algorithm Node

Example:

```
{  
  "type": "algorithm",  
  "name": "ECDSA",  
  "family": "digital_signature",
```

```
"quantum_status": "vulnerable"
}
```

10. Library Node

Example:

```
{
  "type": "library",
  "name": "OpenSSL",
  "version": "3.x",
  "vendor": "OpenSSL Project"
}
```

11. Application Node

```
{
  "type": "application",
  "name": "Payment API",
  "environment": "production",
  "criticality": "critical",
  "owner": "Payments Team"
}
```

12. Certificate Node

```
{
  "type": "certificate",
  "fingerprint": "sha256:...",
  "key_algorithm": "ECDSA",
  "curve": "P-256"
}
```

13. Relationship Types

ECDAT should maintain standardized relationship types.

Examples:

```
USES
DEPENDS_ON
```

IMPLEMENTS
CONTAINS
PROTECTS
PRESENTS
HOSTED_ON
DEPLOYED_IN
OWNED_BY
PART_OF
ISSUED_BY
SIGNED_BY
VERIFIED_BY
CONFIGURED_WITH

14. Evidence on Edges

A very important concept:

Relationships themselves need evidence.

For example:

Application A
└─ **USES** → OpenSSL

ECDAT should know **why** it believes this.

Evidence:

package manifest
container layer
binary import
runtime observation

15. Provenance

Every graph relationship should ideally have:

Source
Scanner
Timestamp
Confidence
Evidence Reference

Example:

```
{
  "relationship": "USES",
  "source": "container_scan",
  "confidence": 0.97,
  "timestamp": "..."}
}
```

16. Multiple Evidence Sources

Suppose:

```
Source scan:
USES OpenSSL
```

```
Container scan:
USES OpenSSL
```

```
Runtime:
USES OpenSSL
```

ECDAT can increase relationship confidence.

17. Graph Construction

The graph should be built from the CBOM.

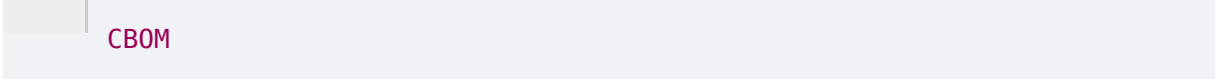
```
Scanners
  ↓
Evidence
  ↓
Canonical CBOM
  ↓
Graph Builder
  ↓
Knowledge Graph
```

Do not make the graph a separate source of truth.

The canonical data model remains authoritative.

18. Graph as a Projection

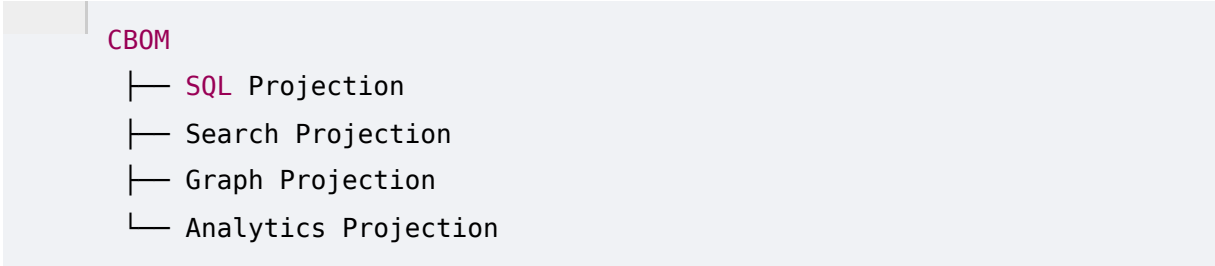
Think of:



CBOM

as the core structured truth.

Then:



CBOM

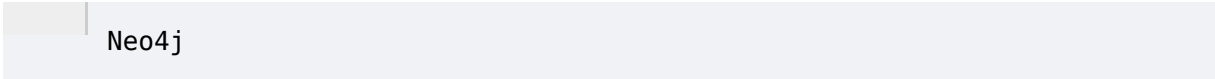
- ├─ SQL Projection
- ├─ Search Projection
- ├─ Graph Projection
- └─ Analytics Projection

This is a strong architecture.

19. Graph Database

A graph database can be used for relationships.

Potential technology:



Neo4j

or other graph-capable systems.

The exact choice depends on deployment requirements.

20. Why Neo4j-Style Graphs Are Attractive

Graph query languages allow questions such as:



Which applications depend on OpenSSL?

or:



Which critical business processes ultimately depend on RSA?

These are naturally expressed as graph traversals.

21. Example Graph Query

Conceptually:

```
Application
  → DEPENDS_ON
  → Library
  → IMPLEMENTS
  → Algorithm
```

Filter:

```
Algorithm = RSA
```

Result:

```
All applications indirectly dependent on RSA.
```

22. Multi-Hop Relationships

The most valuable queries are multi-hop.

Example:

```
Business Process
  ↓
Application
  ↓
Library
  ↓
Algorithm
```

This allows:

```
Which business processes are affected by a vulnerable cryptographic library?
```

23. Blast Radius

This leads to one of ECDAT's strongest features:

Cryptographic Blast-Radius Analysis.

24. What Is Blast Radius?

Blast radius answers:

```
If this cryptographic component changes, what else might be affected?
```

Example:

```
OpenSSL
↓
120 Applications
↓
400 Containers
↓
75 Business Processes
```

That is the blast radius.

25. Why Blast Radius Matters

Without it:

```
Migrate OpenSSL.
```

With it:

```
OpenSSL migration affects:
120 applications
400 containers
75 business processes
18 teams
```

Now management understands the scale.

26. Direct vs Indirect Impact

Direct:

```
Application → OpenSSL
```

Indirect:

```
Business Process
↓
Application
↓
Container
↓
OpenSSL
```

ECDAT should distinguish both.

27. Blast Radius Score

A conceptual model:

$$B = w_1A + w_2C + w_3D + w_4I + w_5R$$

where:

- A = number of affected applications
- C = criticality
- D = dependency depth
- I = infrastructure spread
- R = business impact

The actual weighting should be configurable.

28. Blast Radius Categories

LOW
MEDIUM
HIGH
CRITICAL

Example:

Crypto library
↓
3 development applications

= Low.

But:

Crypto library
↓
1,500 production applications
↓
Payment + Identity + Government systems

= Critical.

29. Algorithm Concentration

The graph can identify:

Which algorithms are used most heavily?

Example:

```
ECDSA:
4,821 assets

RSA:
3,901 assets

AES:
12,421 assets
```

This shows migration concentration.

30. Library Concentration

Example:

```
OpenSSL
↓
3,500 applications
```

This means:

OpenSSL is a cryptographic concentration point.

If OpenSSL is migrated successfully, thousands of applications may benefit.

31. Crypto Single Points of Failure

Graph analysis can reveal:

```
500 applications
↓
One crypto provider
↓
One HSM
```

That represents a potential:

Cryptographic Single Point of Failure.

32. HSM Concentration

Suppose:

```
HSM-01
  ↓
PKI
  ↓
1,200 applications
```

ECDAT should highlight:

```
High concentration
```

because migration or failure of HSM-01 has enormous impact.

33. Certificate Dependency Graph

Certificates create another graph.

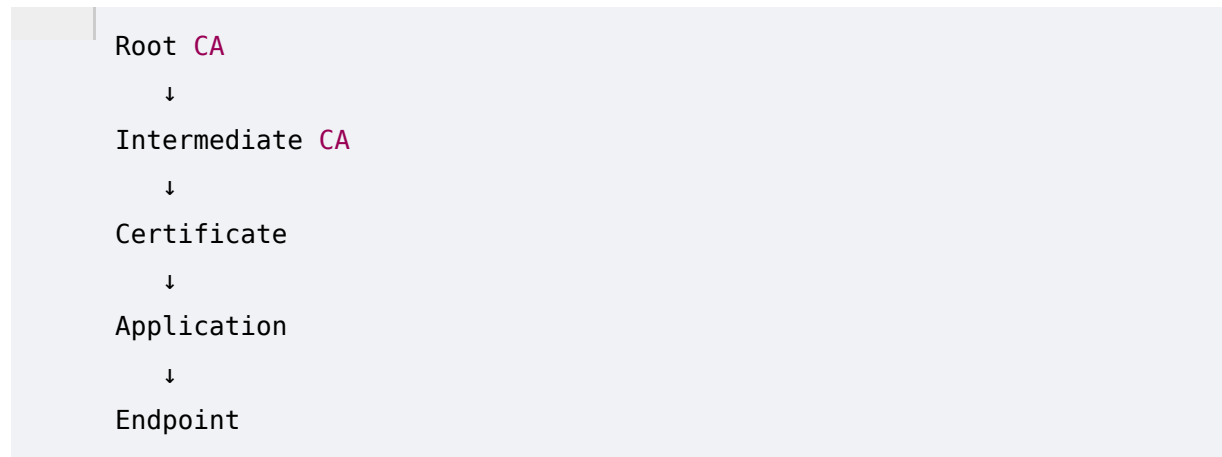
```
Certificate
  ↓
Public Key
  ↓
Algorithm
  ↓
Application
  ↓
Endpoint
```

And:

```
Certificate
  ↓
Issuer
  ↓
CA
  ↓
Root CA
```

34. PKI Graph

Example:

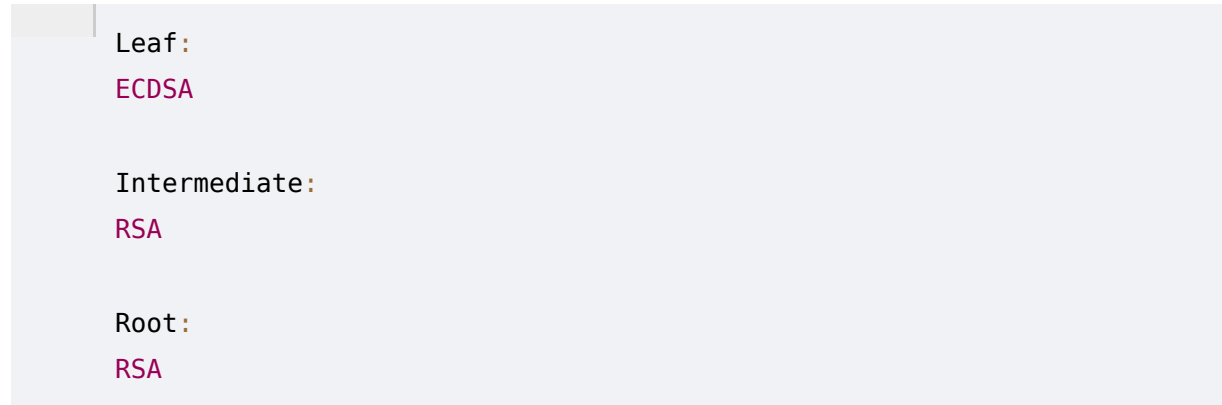


ECDAT can then identify:

Quantum-vulnerable certificate chains.

35. Certificate Chain Risk

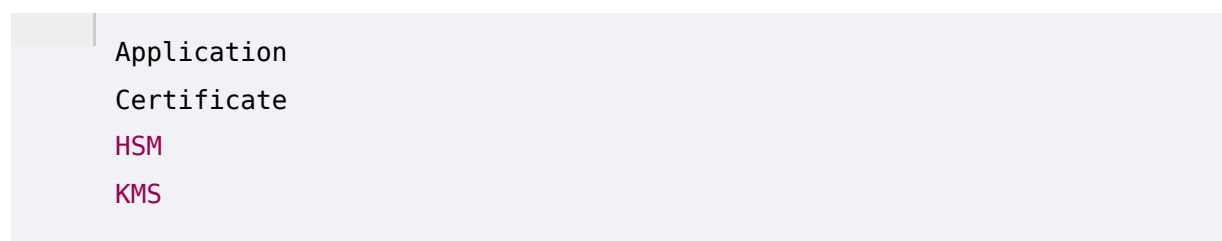
Suppose:



The system should understand the complete chain rather than analyzing only the leaf.

36. Key Dependency Graph

A key can be related to:



Algorithm
Data

Example:

KMS Key
↓
AES-256
↓
Database
↓
Customer Data

37. Data Protection Graph

This becomes powerful.

Sensitive Data
↑
PROTECTED_BY
↑
Encryption
↑
Algorithm
↑
Library
↑
Application

Now the organization can ask:

Which sensitive data depends on quantum-vulnerable cryptography?

38. Long-Lived Data

Combine the graph with the risk engine.

Data
↓
Lifetime = 15 years
↓
Protected by
↓

RSA

↓

Quantum vulnerable

Risk becomes much more meaningful.

39. Mosca Integration

The graph can incorporate Mosca-style relationships:

Data Lifetime

+

Migration Time

vs

Threat Horizon

The graph can show:

Critical Data

↓

RSA

↓

Application

↓

Migration Dependency

40. Migration Dependency Graph

Suppose:

Application

↓

OpenSSL

↓

HSM

↓

PKI

Migration cannot simply happen in application order.

ECDAT can calculate:

```
graph TD; A[HSM upgrade] --> B[PKI upgrade]; B --> C[Library upgrade]; C --> D[Application migration];
```

HSM upgrade
↓
PKI upgrade
↓
Library upgrade
↓
Application migration

41. Migration Critical Path

This is one of the most valuable features.

Example:

```
graph TD; A[HSM Upgrade] --> B[Certificate Infrastructure]; B --> C[Crypto Library]; C --> D[Application]; D --> E[Client Migration];
```

HSM Upgrade
↓
Certificate Infrastructure
↓
Crypto Library
↓
Application
↓
Client Migration

The longest dependency chain becomes:

Migration Critical Path.

42. Migration Bottlenecks

The graph can identify:

```
graph TD; A[Component] --> B[Many dependent systems];
```

Component
↓
Many dependent systems

Example:

```
graph TD; A[Legacy HSM] --> B[900 applications];
```

Legacy HSM
↓
900 applications

This is a migration bottleneck.

43. Bottleneck Score

A conceptual score:

$$M = D \times C \times R$$

where:

- D = number of dependencies
- C = criticality
- R = migration resistance

High M means:

```
Priority:  
Address infrastructure first.
```

44. Graph Centrality

Graph algorithms can identify important nodes.

Potential metrics:

```
Degree Centrality  
Betweenness Centrality  
PageRank-style importance  
Connected Components  
Shortest Paths
```

45. Degree Centrality

A node with many connections:

```
OpenSSL  
↓  
1,000 applications
```

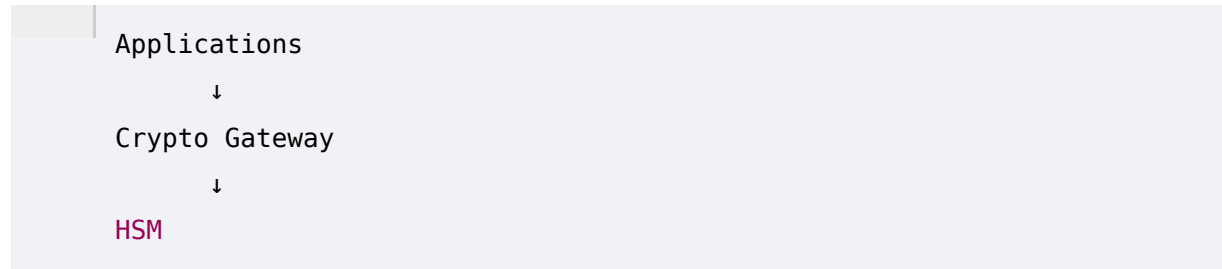
has high degree.

It is an important concentration point.

46. Betweenness Centrality

A node that lies on many dependency paths may be a bottleneck.

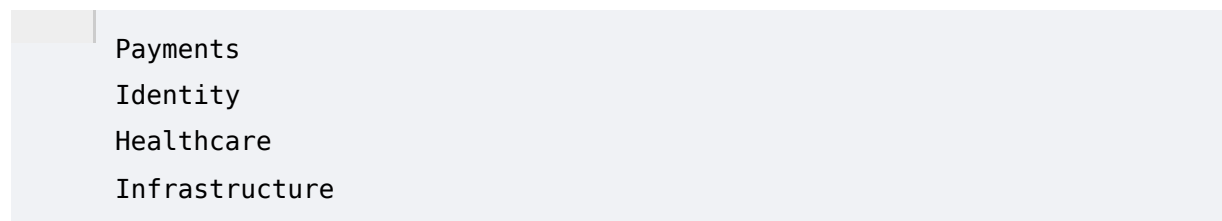
Example:



The gateway has high intermediary importance.

47. Community Detection

The graph can identify clusters:



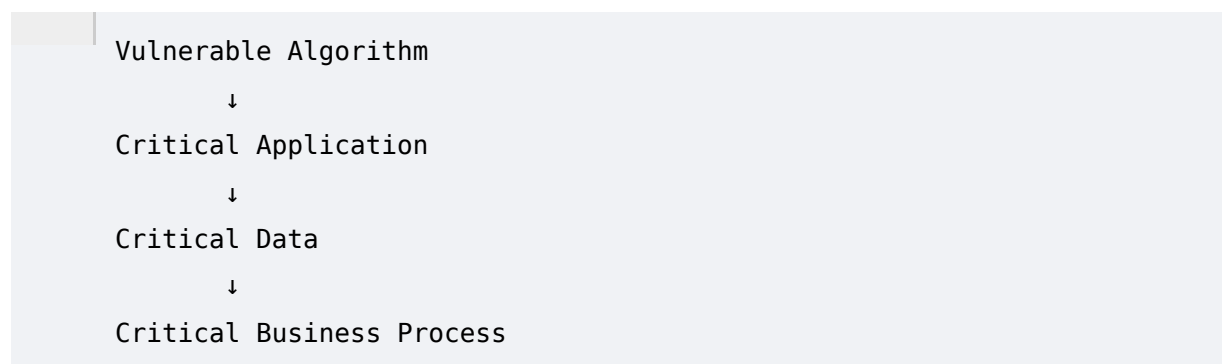
Then ECDAT can determine:

Which business domains have the greatest PQC migration burden?

48. Risk Propagation

Risk can propagate through relationships.

Example:



The graph can calculate business impact.

49. Important Caveat

Risk should not simply propagate mathematically without rules.

For example:

Library contains **RSA**

does not automatically mean:

Every application using library = **RSA** risk.

The graph must distinguish:

Installed
Supported
Configured
Used
Actively Used

50. Usage States

A useful state model:

AVAILABLE
INSTALLED
CONFIGURED
INVOKED
ACTIVE
DEPRECATED
UNUSED
UNKNOWN

51. Example

OpenSSL
├─ supports **RSA**
├─ supports **ECDSA**
└─ supports **AES**

Application uses:

ECDSA

Therefore:

RSA

should not automatically become an active asset for that application.

52. Graph Evidence

Every important edge should be evidence-backed.

Example:

```
Application A
└─ USES → ECDSA
```

Evidence:

```
source file line 42
runtime trace
certificate
```

53. Confidence Propagation

If a relationship is based on weak evidence:

```
String:
"RSA"
```

then:

```
Confidence = Low
```

If supported by:

```
API
+
Certificate
+
Runtime
```

then:

```
Confidence = High
```

54. Graph Versioning

The graph changes constantly.

Therefore ECDAT should support:

```
Current Graph
Historical Graph
Graph Snapshots
Change Events
```

55. Cryptographic Drift

Suppose:

```
Monday:
ECDSA
```

Then:

```
Friday:
ML-DSA
```

ECDAT should show:

```
Cryptographic Change:
ECDSA → ML-DSA
```

This is:

Cryptographic Drift Detection.

56. Drift Types

```
Algorithm Changed
Key Changed
Certificate Changed
Library Changed
Configuration Changed
Protocol Changed
Crypto Provider Changed
```

57. Unauthorized Crypto Drift

Example:

Approved:

AES-256-GCM

Observed:

AES-128-CBC

ECDAT flags:

Policy Drift

58. New Crypto Detection

Developer introduces:

RSA-1024

ECDAT:

New cryptographic asset

↓

Policy evaluation

↓

Forbidden

↓

Alert

59. Graph-Based Investigation

Click:

RSA-1024

ECDAT shows:

RSA-1024

|

├─ Repository A

├─ Application B

├─ Container C

├─ Production Endpoint D

├─ Team E

└─ Business Process F

60. "Why Is This Risky?"

AI can traverse the graph:

```
Algorithm
  ↓
Application
  ↓
Data
  ↓
Criticality
  ↓
Migration Time
```

Then explain:

This algorithm is classified as high priority because it is used by a critical production service that protects long-lived sensitive information and has a migration dependency on a shared PKI component.

61. "What Depends on This?"

User selects:

```
OpenSSL
```

ECDAT answers:

```
1,284 applications
4,210 containers
18 business units
72 business processes
```

62. "What Breaks If I Change This?"

This is an excellent demonstration feature.

User selects:

```
Crypto Library
```

ECDAT computes:

```
Direct dependencies
+
Indirect dependencies
+
Critical services
+
Certificates
+
Business processes
```

and displays:

Potential Blast Radius.

63. Migration Simulation

ECDAT can simulate:

```
IF:
  OpenSSL version changes

THEN:
  Affected applications = X
  Affected containers = Y
  Affected certificates = Z
```

This does not mean the systems will definitely break.

It means:

```
Potential dependency impact.
```

64. What-If Analysis

Another powerful feature:

```
What if we migrate all RSA signatures to ML-DSA?
```

ECDAT can estimate:

```
Affected applications
Affected certificates
Affected protocols
```

Affected libraries
Migration blockers

65. Graph-Based Prioritization

Combine:

Risk

+

Centrality

+

Criticality

+

Blast Radius

+

Migration Complexity

Then calculate:

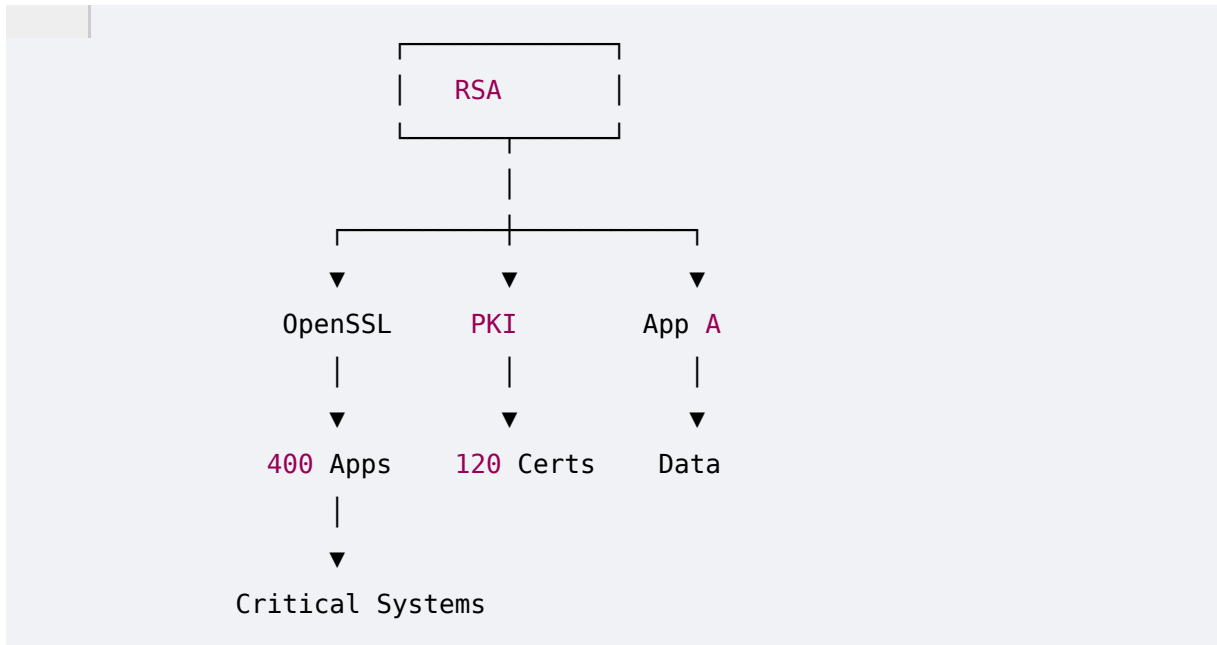
Strategic Migration Priority.

66. Example Priority

Asset	Quantum Risk	Blast Radius	Criticality	Migration Difficulty	Priority
RSA PKI	High	Very High	Critical	High	P1
ECDSA API	High	Medium	Critical	Medium	P1
AES-256	Low	High	Critical	Low	P3
Legacy SHA-1	Medium	Medium	Medium	Low	P2

67. Graph Dashboard

A powerful visualization:



68. Risk Heatmap

Graph nodes can be colored by:

Quantum Risk

and sized by:

Blast Radius

Therefore:

Large + Red

means:

High-risk, high-impact asset.

69. Migration Heatmap

Alternative visualization:

Node Size: Number of dependencies

Node Color:
Migration difficulty

This identifies bottlenecks immediately.

70. Business Heatmap

Another view:

Business Unit
↓
Crypto Assets
↓
Criticality
↓
Migration Readiness

Example:

Payments:
High exposure

Identity:
Very High exposure

HR:
Moderate

Development:
Low

71. Graph-Based Search

Users can search:

"Show critical applications using ECDSA."

or:

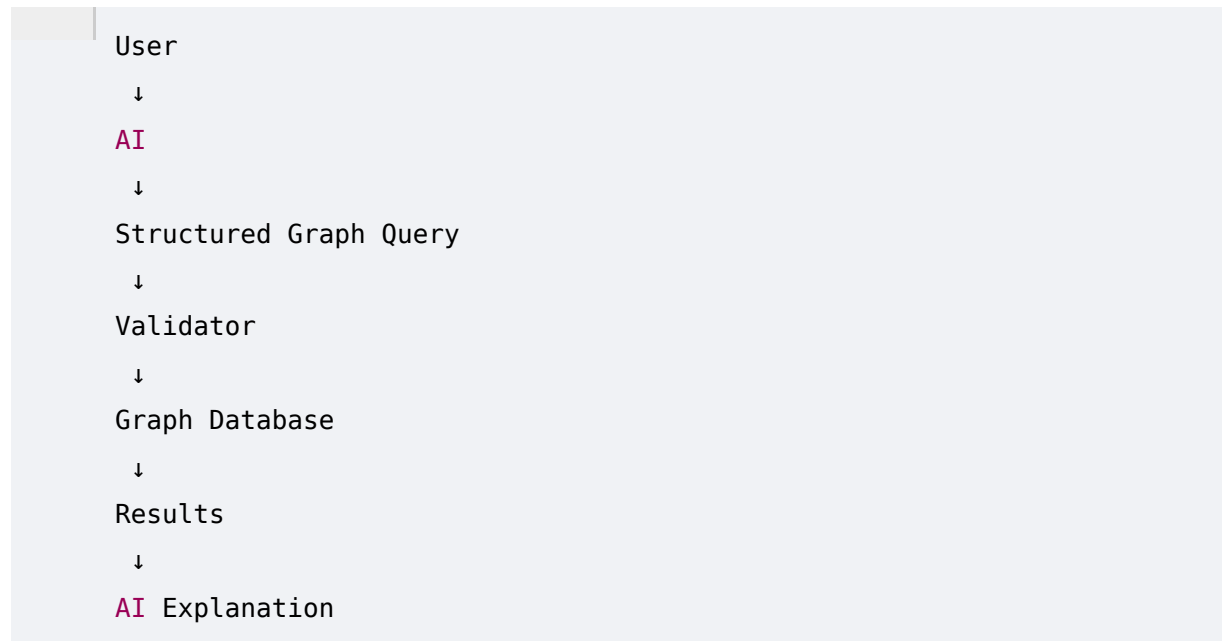
"Show applications dependent on OpenSSL 1.x."

or:

"Show certificates with quantum-vulnerable signature algorithms."

72. Graph Query Layer

Natural language:

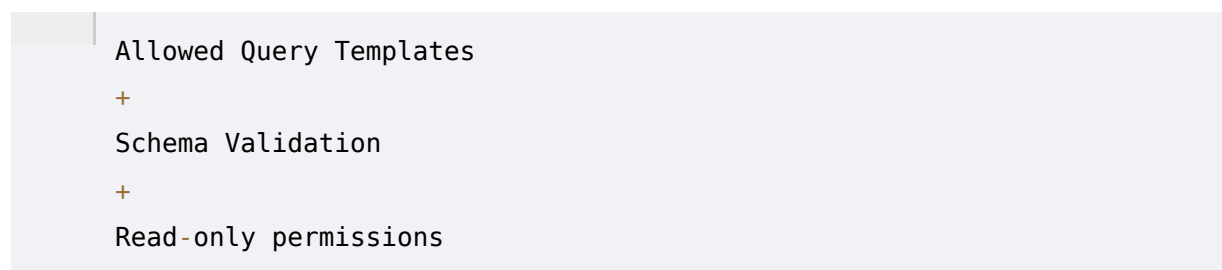


This extends Phase 14's AI architecture.

73. Graph Query Safety

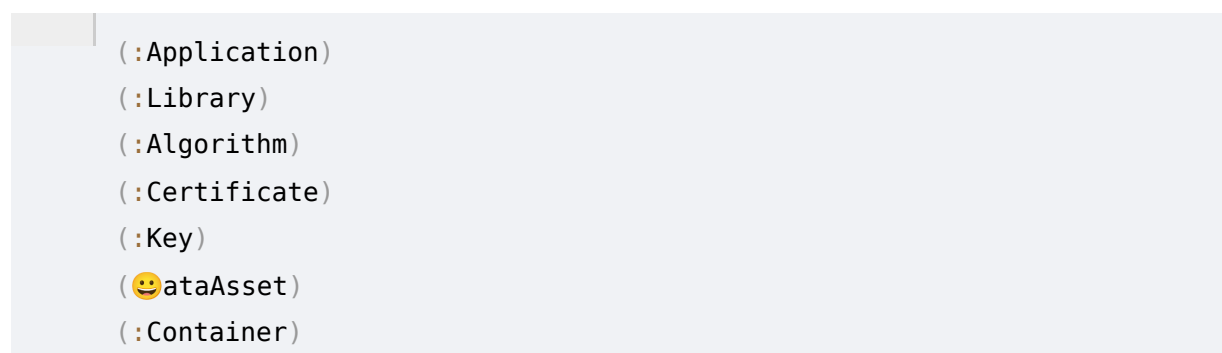
Never allow the LLM to generate unrestricted database commands.

Use:



74. Graph Data Model

A simplified model:



```
(:Endpoint)
(:BusinessProcess)
(:Team)
```

Relationships:

```
(:Application) - [:USES] -> (:Library)

(:Library) - [:IMPLEMENTS] -> (:Algorithm)

(:Application) - [:PRESENTS] -> (:Certificate)

(:Certificate) - [:USES] -> (:Algorithm)

(:Application) - [:PROTECTS] -> (😊ataAsset)

(:Application) - [:OWNED_BY] -> (:Team)

(:Application) - [:SUPPORTS] -> (:BusinessProcess)
```

75. Relationship Metadata

Example:

```
USES {
  confidence: 0.95,
  evidence_id: "E123",
  discovered_at: "...",
  source: "binary-scanner"
}
```

76. Graph Integrity

ECDAT must prevent impossible relationships.

Example:

```
Certificate
└─ USES → Python Function
```

is invalid.

Use schema constraints.

77. Graph Data Quality

Track:

```
Missing Owners
Unknown Applications
Unresolved Dependencies
Low-confidence Edges
Conflicting Relationships
Orphaned Assets
```

78. Orphan Detection

Example:

```
Algorithm:
RSA

No application
No library
No source
```

ECDAT should classify:

```
Orphaned / Unresolved Asset
```

and investigate its origin.

79. Duplicate Identity

Same application may appear as:

```
payment-api
payment-api-prod
payment_service
payment-api:v4
```

Entity resolution is necessary.

80. Entity Resolution

Use:

```
Repository URL
Artifact Digest
Application ID
Package Coordinates
Certificate Fingerprint
Cloud Resource ID
```

to establish identity.

AI can assist with ambiguous matching, but deterministic identifiers should dominate.

81. Temporal Graph

ECDAT can eventually represent:

```
2026:
RSA

2027:
Hybrid

2028:
PQC
```

This allows migration tracking.

82. Historical Investigation

User asks:

```
When did this application stop using RSA?
```

ECDAT traverses graph snapshots:

```
Jan:
RSA

Apr:
RSA + PQC

Aug:
PQC
```

83. Migration Verification

After a migration:

```
Old:  
RSA
```

ECDAT rescans.

Graph should change:

```
Application  
├─ USED RSA  
└─ USES ML-DSA
```

The historical edge remains for audit.

84. Graph Audit Trail

Never simply delete relationships.

Instead:

```
ACTIVE  
RETIRED  
REPLACED  
UNKNOWN
```

This maintains history.

85. Dependency Depth

Example:

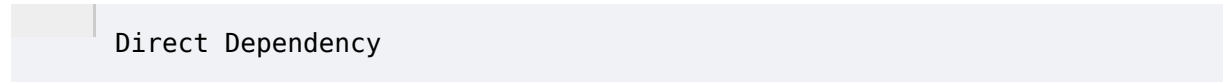
```
Business Process  
  ↓ 1  
Application  
  ↓ 2  
Container  
  ↓ 3  
Library  
  ↓ 4  
Algorithm
```

ECDAT can calculate dependency depth.

86. Deep Dependencies

A vulnerability buried five layers deep may still affect production.

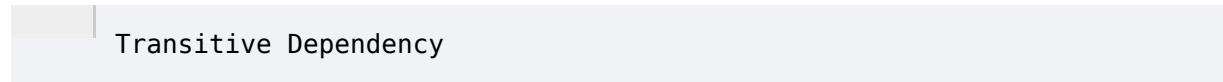
Therefore:



Direct Dependency

A diagram showing a light blue rectangular box with a vertical line on its left side. The text "Direct Dependency" is centered within the box.

and:



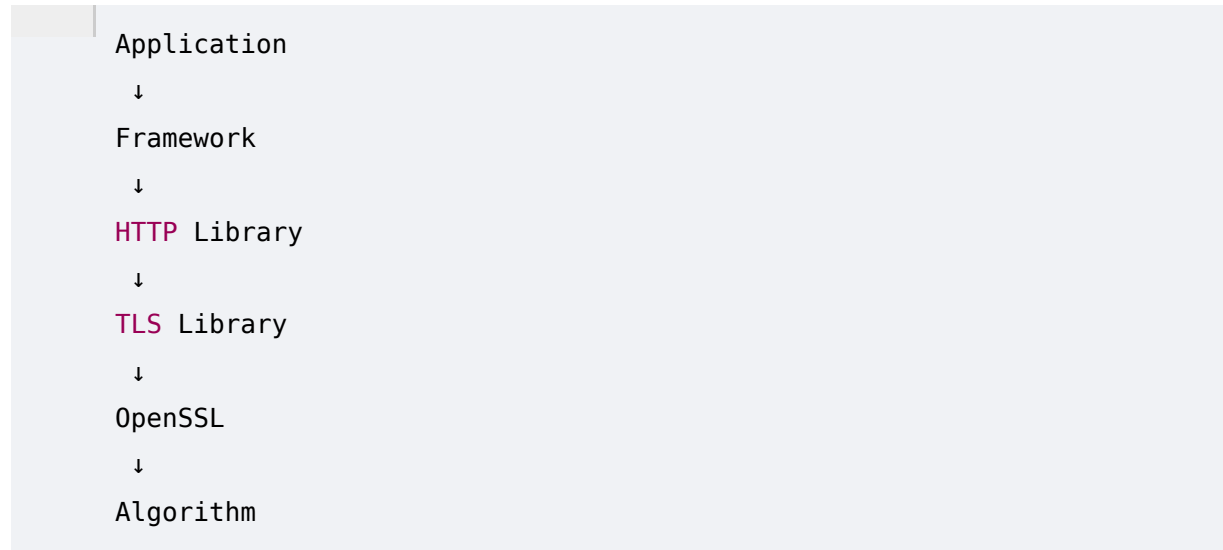
Transitive Dependency

A diagram showing a light blue rectangular box with a vertical line on its left side. The text "Transitive Dependency" is centered within the box.

must be distinguished.

87. Transitive Crypto Dependencies

Example:

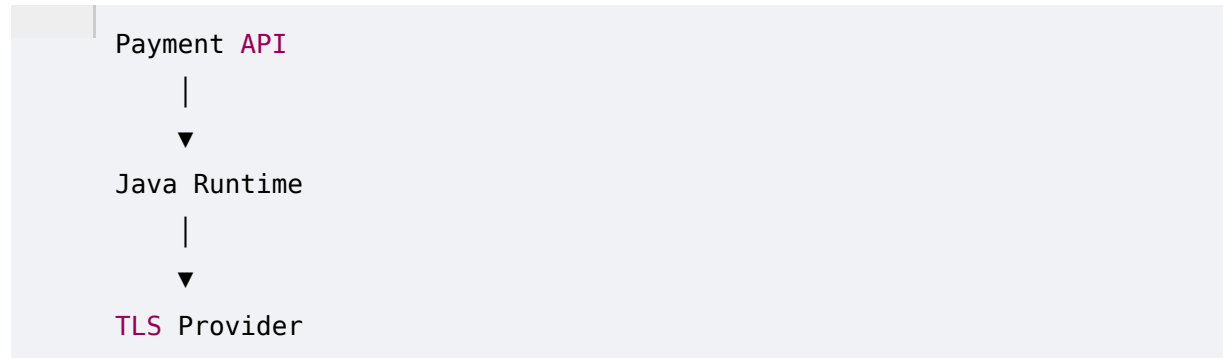


```
graph TD; Application --> Framework; Framework --> HTTP_Library[HTTP Library]; HTTP_Library --> TLS_Library[TLS Library]; TLS_Library --> OpenSSL; OpenSSL --> Algorithm;
```

A diagram showing a vertical sequence of components connected by downward arrows. The components are: Application, Framework, HTTP Library (with "HTTP" in pink), TLS Library (with "TLS" in pink), OpenSSL, and Algorithm.

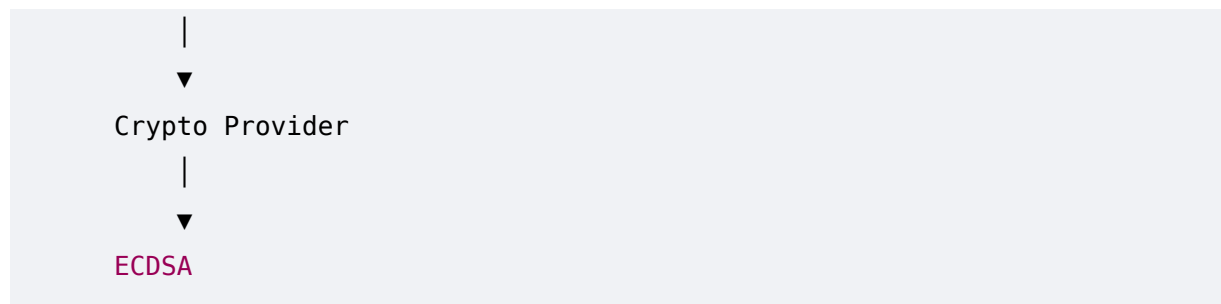
This is a cryptographic dependency chain.

88. Dependency Graph Example



```
graph TD; Payment_API[Payment API] --> Java_Runtime[Java Runtime]; Java_Runtime --> TLS_Provider[TLS Provider];
```

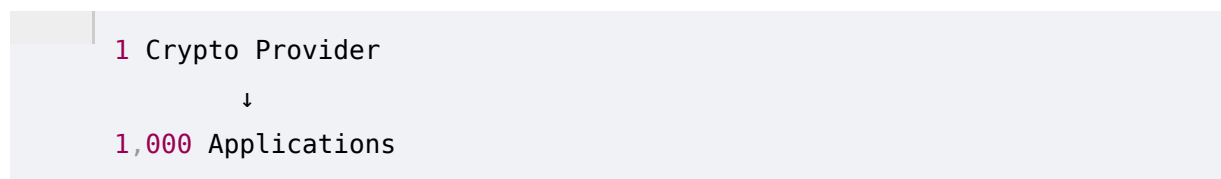
A diagram showing a vertical sequence of components connected by downward arrows. The components are: Payment API (with "API" in pink), Java Runtime, and TLS Provider (with "TLS" in pink).



Now migration impact becomes visible.

89. Shared Components

Suppose:



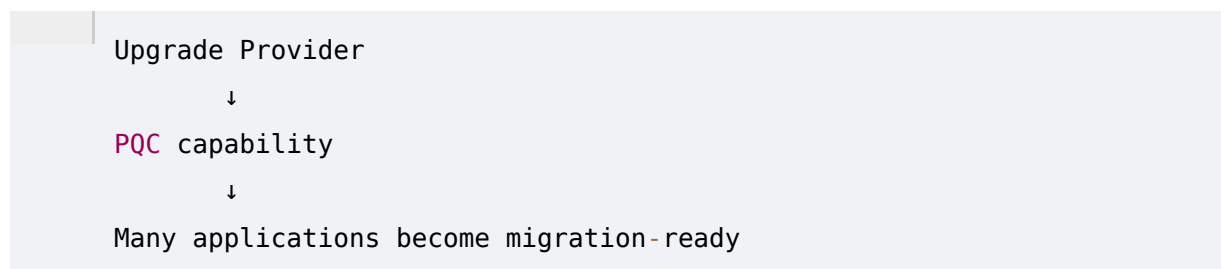
The provider is strategically important.

ECDAT should highlight:

Shared Cryptographic Infrastructure.

90. Shared Component Strategy

Rather than migrating 1,000 applications individually:



This is a major enterprise optimization.

91. Migration Leverage

ECDAT can identify:

Which migration action produces the largest risk reduction?

For example:

Upgrade Crypto Provider

↓

2,000 assets become PQC-capable

This has high leverage.

92. Risk Reduction per Effort

A useful strategic metric:

$$L = \frac{\text{Risk Reduction}}{\text{Migration Effort}}$$

High L :

High-leverage migration.

93. Portfolio Optimization

ECDAT can rank:

Migration A

Risk reduction: 90

Effort: 10

versus:

Migration B

Risk reduction: 50

Effort: 40

Migration A becomes a priority.

94. Graph + Risk + Recommendation

The real ECDAT intelligence loop becomes:

GRAPH

↓

What depends on what?

↓

RISK

↓

What is dangerous?

↓

RECOMMENDATION

↓

What should change?

↓

GRAPH

↓

What will be affected?

95. Graph + AI

AI can explain graph structures.

Example:

"Why is this HSM migration considered critical?"

AI retrieves:

HSM

↓

PKI

↓

Certificates

↓

Applications

↓

Business Processes

and produces a grounded explanation.

96. Graph + Quantum Risk

Graph enables:

Quantum Vulnerable Algorithm

↓

Applications

↓

Data

↓

Business Impact

This makes quantum risk business-aware.

97. Graph + Mosca

Graph can connect:

```
Data Lifetime
Migration Dependency
Algorithm Risk
```

Example:

```
Data lifetime:
15 years

Migration:
5 years

Quantum threat horizon:
10 years

Result:
Critical
```

98. Graph + PQC Recommendations

Recommendation can be propagated through dependencies.

Example:

```
Library X
↓
1,200 apps
```

ECDAT may recommend:

```
Prioritize upgrading Library X to a PQC-capable version before application-level
migrations.
```

99. Graph + Crypto Agility

The graph can identify:

```
Applications with
hardcoded algorithms
```

and:

Applications using
crypto abstraction

Then calculate:

Crypto-agility maturity

by business unit.

100. Executive View

The graph ultimately supports questions like:

What are our top five cryptographic migration bottlenecks?

Example output:

1. Legacy **HSM** infrastructure
 2. Shared OpenSSL dependency
 3. Legacy **PKI**
 4. **TLS** client compatibility
 5. Embedded firmware
-

101. Technical View

Security engineers can drill down:

Legacy **HSM**
↓
Partition **3**
↓
RSA Keys
↓
PKI
↓
Certificates
↓
Applications

102. Developer View

Developers see:

```
Your Application
  ↓
ECDSA usage
  ↓
File
  ↓
Function
  ↓
Crypto library
  ↓
Migration recommendation
```

103. Management View

Management sees:

```
Critical Assets:
342

PQC Migration:
28%

Migration Blockers:
17

Top Bottleneck:
Legacy PKI

Estimated Impact:
High
```

104. Graph API

Possible API:

```
GET /api/graph/assets/{id}
GET /api/graph/dependencies/{id}
GET /api/graph/blast-radius/{id}
```

```
GET /api/graph/paths
GET /api/graph/impact-analysis
```

105. Blast Radius API

Example:

```
{
  "asset": "openssl",
  "direct_dependents": 320,
  "indirect_dependents": 1284,
  "critical_applications": 74,
  "business_processes": 32,
  "teams": 18,
  "blast_radius": "CRITICAL"
}
```

106. Graph Investigation API

Example:

```
GET /api/graph/path?
from=algorithm:rsa
to=data:sensitive
```

Result:

```
RSA
↓
Library
↓
Application
↓
Database
↓
Sensitive Data
```

107. Security Considerations

The graph itself can contain highly sensitive information.

It may reveal:

- Internal Architecture
- Business Systems
- Dependencies
- Security Infrastructure
- HSM Locations
- PKI Relationships

Therefore:

- RBAC
- Encryption
- Audit
- Access Control
- Tenant Isolation

are mandatory.

108. Role-Based Graph Access

Example:

Developer

Can see:

- Own applications

Security Analyst

Can see:

- Enterprise crypto graph

Executive

Can see:

- Aggregated business impact

Infrastructure Admin

Can see:

- Infrastructure relationships

109. Graph Privacy

Sensitive nodes may be masked:

```
HSM-001
```

instead of revealing exact infrastructure details to unauthorized users.

110. Performance

Graph queries can become expensive at enterprise scale.

Use:

```
Indexes  
Caching  
Materialized Views  
Graph Partitioning  
Query Limits  
Precomputed Metrics
```

111. Precomputed Metrics

For example:

```
Blast Radius  
Centrality  
Criticality  
Migration Difficulty
```

can be calculated periodically rather than on every dashboard request.

112. Graph Event Model

When something changes:

```
New Library
```

emit:

```
LIBRARY_DISCOVERED
```

When algorithm changes:

```
ALGORITHM_CHANGED
```

When dependency changes:

```
DEPENDENCY_CHANGED
```

113. Event-Driven Graph Updates

```
Scanner  
  ↓  
Finding  
  ↓  
Event Bus  
  ↓  
Graph Updater  
  ↓  
CBOM
```

This enables near-real-time updates.

114. Graph Consistency

If:

```
CBOM says:  
Application uses ECDSA
```

but:

```
Graph says:  
Application uses RSA
```

ECDAT should detect inconsistency.

The CBOM remains the canonical source.

115. Graph Rebuild

ECDAT should support rebuilding the graph from CBOM:

```
CBOM  
  ↓  
Graph Builder
```

↓
Fresh Graph

This provides recovery from graph corruption or migration.

116. Graph Snapshots

Take snapshots:

2026-08
2026-09
2026-10

Then compare:

Graph Diff

117. Graph Diff

Example:

Added :
ML-DSA

Removed :
RSA

Changed :
OpenSSL version

This can become part of migration verification.

118. Cryptographic Posture Over Time

Graph snapshots enable:

Quantum Vulnerable Assets

2026 → 8,421
2027 → 6,900
2028 → 4,100

2029 → 1,200

2030 → 200

This provides measurable migration progress.

119. Executive KPI

A very powerful KPI:

Quantum Vulnerable Asset Reduction.

Example:

Baseline:

8,421

Current:

5,302

Reduction:

37%

120. Migration Progress

Another KPI:

PQC-ready applications:

24%

Hybrid-enabled:

31%

Legacy:

45%

121. Graph-Based Risk Reduction

ECDAT can calculate:

Before migration:

Risk = 87

After migration:

Risk = 41

This provides evidence that migration is working.

122. Phase 16 Core Requirements

The graph system should support:

Nodes

- Applications
- Libraries
- Algorithms
- Keys
- Certificates
- Containers
- Infrastructure
- Data
- Teams
- Business processes

Edges

- Uses
- Depends on
- Implements
- Protects
- Presents
- Hosted on
- Owned by
- Issued by

Analytics

- Blast radius
- Centrality
- Dependency depth
- Migration bottlenecks
- Risk propagation

- Concentration
- Critical path

Time

- Historical graph
- Drift
- Graph snapshots
- Migration verification

123. The Most Important Insight

The biggest leap here is:

ECDAT should not treat cryptographic assets as isolated findings.

Cryptography exists inside systems.

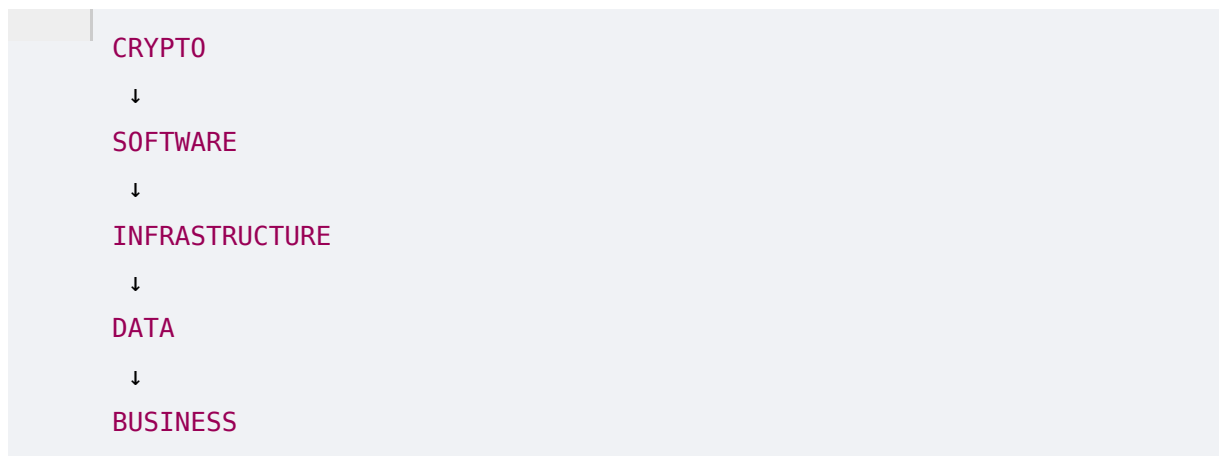
Systems exist inside infrastructure.

Infrastructure supports applications.

Applications protect data.

Data supports business processes.

Therefore:



must ultimately be represented.

124. ECDAT Becomes a Dependency Intelligence

Platform

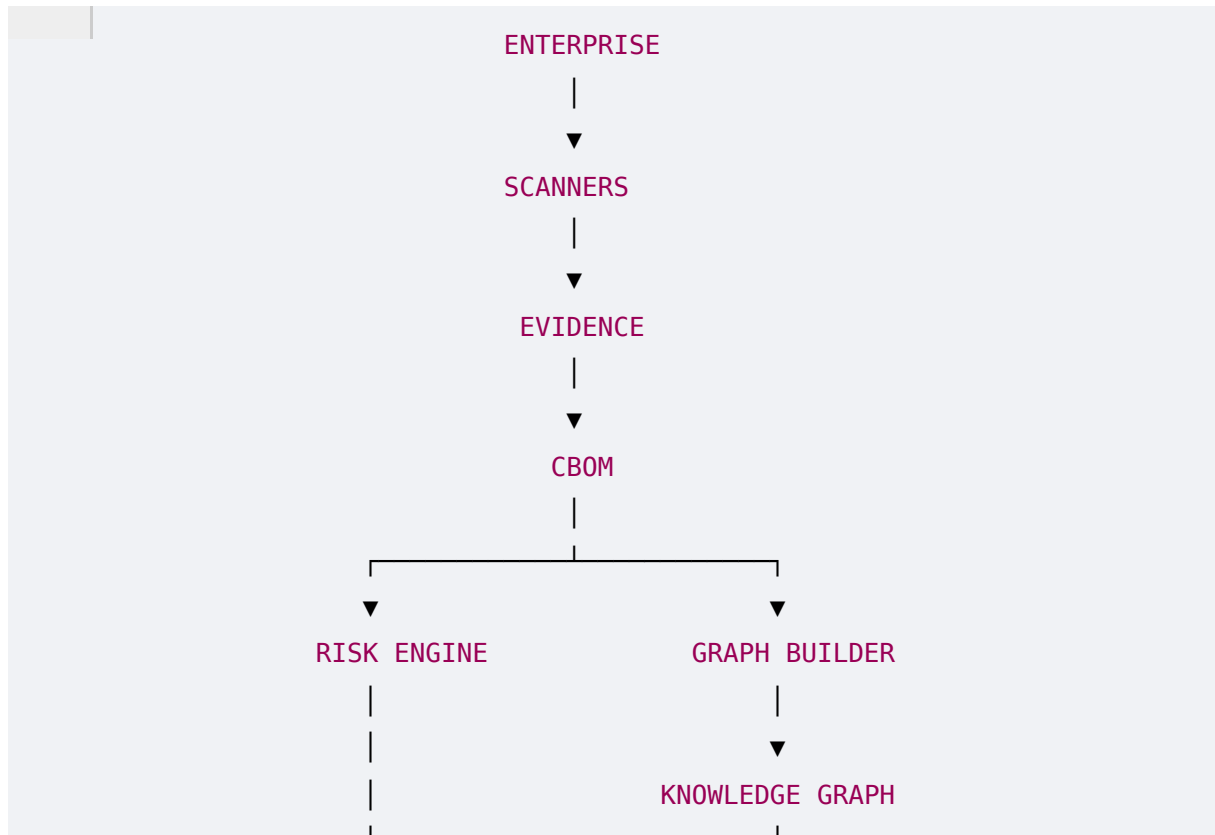
At this stage, ECDAT is no longer merely:

It is becoming:

Cryptographic Dependency Intelligence.

And this is much more powerful.

125. Phase 16 Final Architecture



126. Phase 16 Final Takeaway

The core question ECDAT should answer is no longer simply:

"What cryptography do we have?"

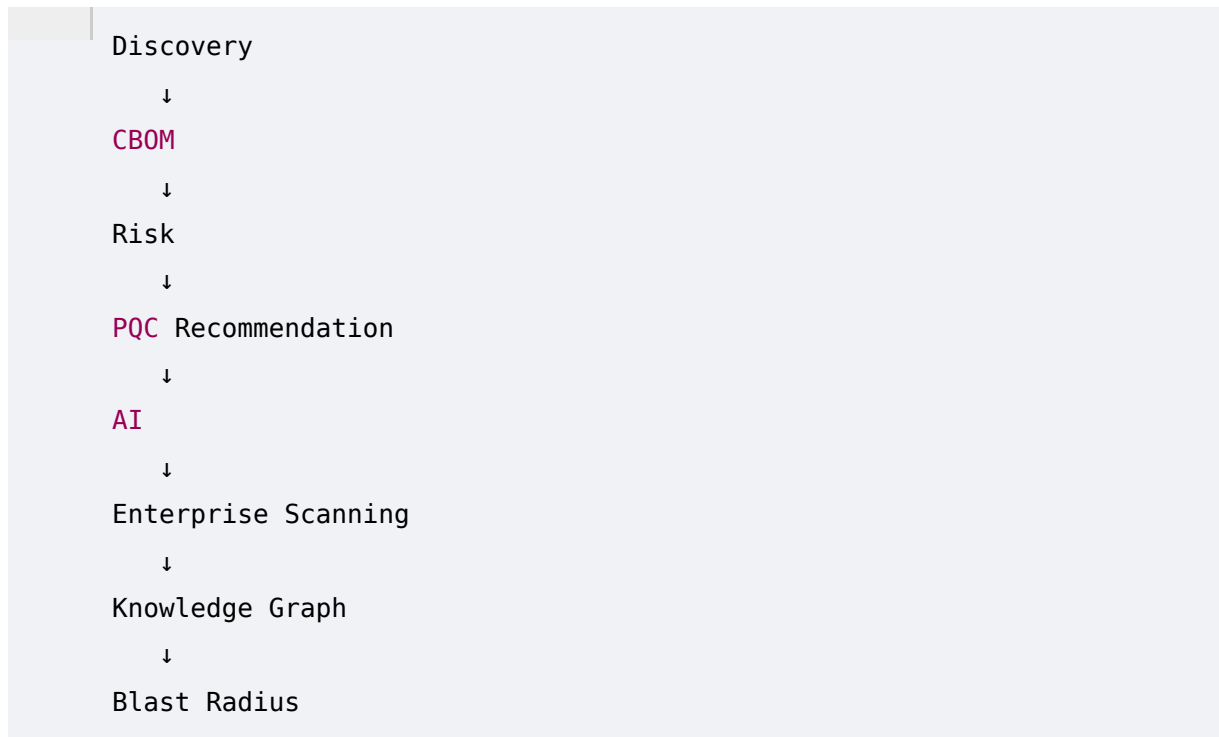
It should answer:

"Where is it, what depends on it, what does it protect, how important is it, what happens if it changes, and what is the most efficient migration path?"

That is the difference between a scanner and an **enterprise cryptographic intelligence platform**.

PHASE 17 PREVIEW — ENTERPRISE RISK DASHBOARD, QUANTUM READINESS SCORE & EXECUTIVE DECISION SYSTEM

Now that ECDAT has:



the next question becomes:

How does a security team actually operate this system?

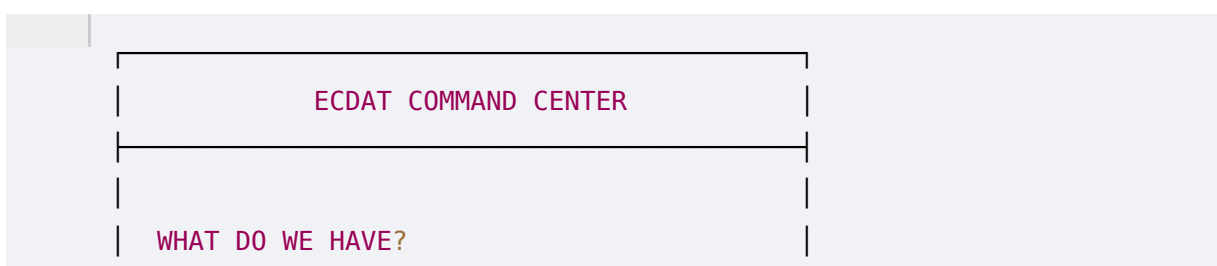
Phase 17 will design the complete **ECDAT Command Center**.

It will cover:

- Executive dashboard
- Security analyst dashboard
- Developer dashboard
- Infrastructure dashboard
- Quantum readiness score
- Enterprise crypto posture score
- Application risk score
- Business-unit risk
- Migration readiness
- Migration progress

- Risk heatmaps
- Asset explorer
- CBOM explorer
- Graph explorer
- Vulnerability explorer
- Certificate explorer
- Algorithm explorer
- PQC recommendation center
- Migration planner
- Critical-path visualization
- Blast-radius dashboard
- Compliance dashboard
- Policy violations
- Crypto-agility dashboard
- Executive reports
- Technical reports
- Scheduled reporting
- PDF/CSV/JSON export
- SIEM integration
- Alerting
- Notifications
- Risk thresholds
- KPI design
- Drill-down architecture
- UX hierarchy
- Role-based views
- Multi-tenant architecture
- Audit dashboards

The central dashboard will ultimately answer four questions:



	→ Cryptographic Inventory	
	HOW EXPOSED ARE WE?	
	→ Quantum Risk	
	WHAT SHOULD WE DO?	
	→ PQC Recommendations	
	HOW FAR HAVE WE PROGRESSED?	
	→ Migration Readiness	

And the ultimate executive KPI will be:

Quantum Readiness Score

A single score backed by the complete evidence graph, rather than an arbitrary AI-generated number.

Phase 16 complete.